

Sezione 4/6 — Sync protocol e API contract

Push/pull, optimistic concurrency, conflict resolution, contratto REST per EM Datacenter

4.1 Modello di concorrenza

Optimistic concurrency a granularità di **grafo intero** (non per-nodo). Ogni grafo pubblicato ha:

- `graph_id` — UUID v7, generato al primo publish
- `revision` — UUID v7 monotono, bumpato a ogni push accettato
- `parent_revision` — la revision da cui è derivata questa
- `checksum` — SHA-256 del GraphML canonicalizzato (per integrity check)

Il client pyarchinit conserva localmente: `last_known_remote_revision` in `QSettings` per progetto.

PERCHÉ GRAFO-INTERO E NON PER-NODO

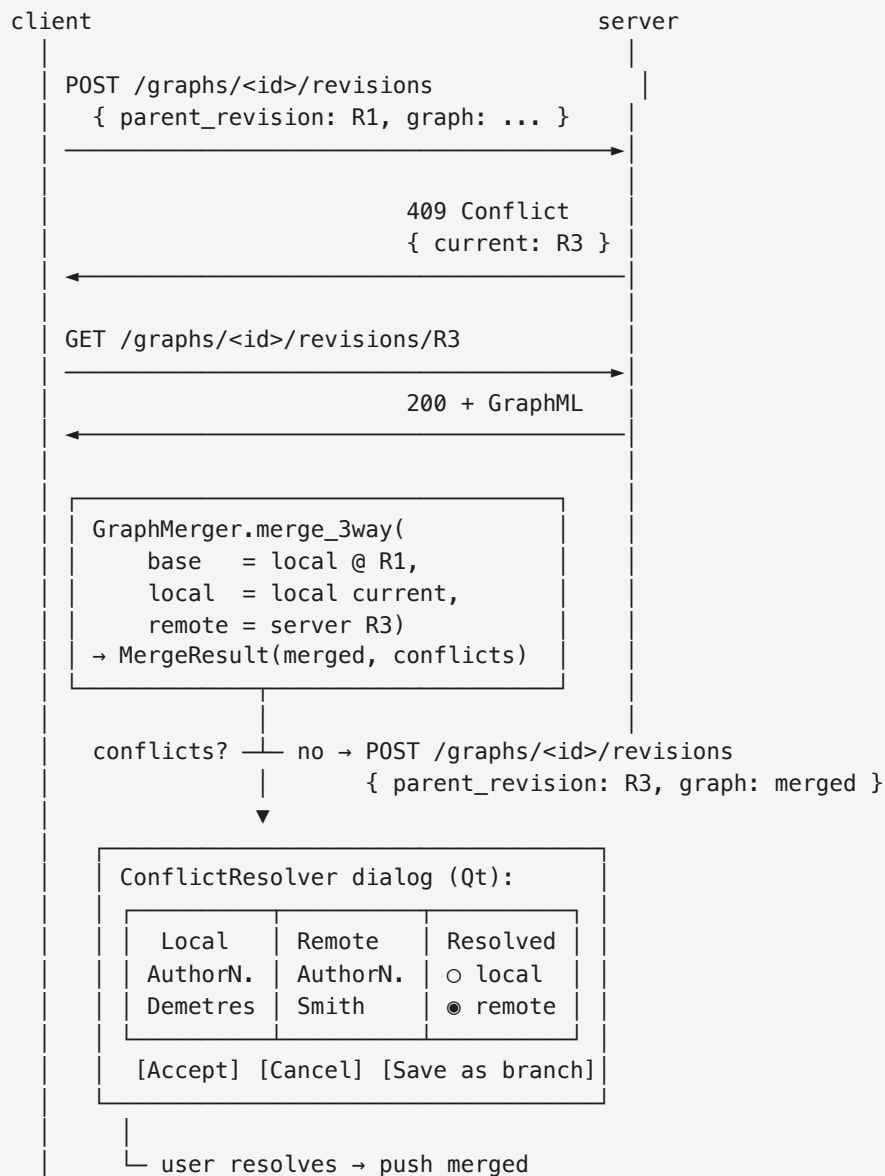
Un Harris matrix è coerente solo come unità: cambiare un rapporto può rendere invalido un altro. Pubblicare nodo-per-nodo richiederebbe transazioni distribuite o un solver di vincoli sul server. Grafo-intero è atomico, predicibile, e i payload restano sotto il MB anche per scavi grandi (testato: 1500 US ≈ 600 KB GraphML).

4.2 Operazioni di sync

Op	Trigger	Cosa fa
<code>pull()</code>	Manuale (pulsante) o all'apertura del progetto QGIS	Scarica HEAD remoto se la revisione differisce dal <code>last_known_remote_revision</code> . Confronta con strato paradata locale → 3-way merge tramite <code>GraphMerger</code> . Se conflitto → <code>ConflictResolver</code> dialog. Aggiorna <code>paradata.graphml</code> e <code>last_known_remote_revision</code> .
<code>push()</code>	Manuale ("Publish") o evento "ready to publish"	Combina strato stratigrafico (proiettato) + paradata locale → calcola checksum. Invia con <code>parent_revision = last_known_remote_revision</code> . Se il server ha avanzato → 409 Conflict → si entra nel flusso di pull+merge+retry.
<code>diff()</code>	Pre-publish, dry-run	Mostra all'utente il diff (nodi aggiunti/rimossi/cambiati) prima di confermare il push.
<code>fetch_metadata()</code>	Background, leggero	HEAD-only request: <code>GET /graphs/<id>/HEAD</code> restituisce solo <code>{revision, checksum, updated_at, updated_by}</code> . Aggiorna l'indicatore "remote ahead/behind" nella status bar.

4.3 Conflict resolution

Flusso conflitto su push



Policy per tipo di nodo (auto-merge dove sicuro, dialog dove serve):

Tipo nodo	Policy
StratigraphicUnit (US/USM/USD/SF)	Sempre dialog — sono dati primari, mai auto-merge silenzioso
EpochNode	Auto-merge se solo timestamp cambia, dialog se cambia label/range temporale
AuthorNode	Last-writer-wins (campo <code>updated_at</code>), notifica passiva
LicenseNode, EmbargoNode	Server vince sempre (compliance / gestione centralizzata)
USVn / USVs / VSF	Dialog (sono interpretazioni, conflitti sono semanticamente significativi)
DocumentNode	Merge per-property (URI immutabile, metadata last-writer-wins)

4.4 REST API contract per EM Datacenter

Questo è il contratto che il team EM Datacenter deve implementare. Versionato come `/api/v1/` .

Endpoint	Metodo	Scopo
<code>/api/v1/graphs</code>	GET	Lista grafi pubblicati (filtri: site, author, embargo_status, period). Paginato.
<code>/api/v1/graphs</code>	POST	Crea un nuovo graph_id. Body: <code>{site, area, owner, license_id}</code> . Restituisce <code>{graph_id, initial_revision}</code> .
<code>/api/v1/graphs/{id}/HEAD</code>	GET	Metadati head: <code>{revision, parent_revision, checksum, updated_at, updated_by, node_count}</code> . Cheap.
<code>/api/v1/graphs/{id}/revisions/{rev}</code>	GET	Scarica GraphML completo della revisione. <code>If-None-Match</code> header supportato (304).
<code>/api/v1/graphs/{id}/revisions</code>	POST	Push nuova revisione. Body: <code>{parent_revision, graphml, checksum, message}</code> . Restituisce 201+nuovo revision oppure 409+current revision.
<code>/api/v1/graphs/{id}/revisions</code>	GET	Cronologia revisioni (per "git log" view). Paginata.
<code>/api/v1/vocabularies/{name}</code>	GET	Restituisce JSON di un vocabolario (<code>units</code> , <code>edges</code> , <code>paradata</code> , <code>cidoc</code>). <code>If-Modified-Since</code> supportato.
<code>/api/v1/vocabularies</code>	GET	Lista vocabolari disponibili con <code>updated_at</code> .

Status codes pertinenti:

- `200 OK` — read success
- `201 Created` — push accepted, body contiene `{revision, checksum}`
- `304 Not Modified` — pull no-op
- `401/403` — auth issue
- `409 Conflict` — parent_revision stale, body contiene `{current_revision}`
- `422 Unprocessable Entity` — graph fails schema validation, body lista violazioni

- 429 Too Many Requests — rate limit, Retry-After header

4.5 Auth

Fase 1 (MVP): **API key** in `Authorization: Bearer <key>`. Salvata in `QSettings` con flag `encrypted`. Configurazione via dialog "PyArchInit → Settings → Datacenter".

Fase 2: **OAuth 2.0 device flow** (per integrazione con SSO istituzionale). Il `DatacenterClient` ha già astrazione su `IAuthProvider` per supportare entrambi senza riscrivere il client.

4.6 Offline e retry

Situazione	Comportamento
Push offline	Salva payload + parent_revision in <code>~/.cache/pyarchinit/sync_queue.sqlite</code> . Indicatore "pending" in status bar. Retry al prossimo network event con backoff esponenziale (2s, 4s, 8s, 16s, 30s, max 60s).
Pull offline	Usa l'ultimo GraphML cached. Status bar mostra "stale (last sync: 2h ago)".
Conflict in coda offline	Quando si torna online, il push fallisce con 409 → entra nel flusso ConflictResolver. Niente è perso.
Server 5xx	Retry con backoff. Se persiste > 5 minuti, notifica utente e mantiene in coda.

4.7 Variante GraphDB / triple-store

Il `GraphDBBackend` usa lo stesso `SyncEngine` ma traduce diversamente:

Operazione EM	Equivalente GraphDB
POST <code>/graphs/{id}/revisions</code>	SPARQL UPDATE dentro un named graph <code><urn:pyarchinit:graph:{id}:rev:{rev}></code> . Versionamento via named graphs.
GET <code>/graphs/{id}/revisions/{rev}</code>	CONSTRUCT { ?s ?p ?o } FROM <code><urn:...:rev:{rev}></code> WHERE { ?s ?p ?o }
HEAD detection	Property <code>pa:isHead "true"</code> sul named graph corrente
Conflict	Lock+check via SPARQL: <code>ASK { GRAPH ?g { ?g pa:revision <{parent_rev}> ; pa:isHead true } }</code>

Il GraphML lato pyarchinit si converte in RDF Turtle/N-Triples al volo usando il mapping CIDOC-CRM esposto da `VocabProvider.get_cidoc_mapping()`. Il `GraphDBBackend` incapsula la conversione, il `SyncEngine` non se ne accorge.

4.8 Idempotenza

- Ogni push include un `request_id` (UUID v7) header. Se il server riceve lo stesso `request_id` due volte (es. retry dopo timeout), restituisce 200 con la revisione già creata invece di crearne una nuova.
- Il client persiste `request_id` nella coda offline finché non riceve conferma 2xx, così il retry post-crash non duplica.